University of Piraeus

Department of Digital Systems

Level: Postgraduate Program of Study

"ScamAI - Fraud Detection and Automated Response (Scambaiting) System with Artificial Intelligence"

Course	Advanced Cybersecurity and Artificial Intelligence Topics
Work	MSc Semester Project 1
Team	Vassiliadou Chrysoula, Vation Adriana, Kalaitzidis Ioannis, Kapnia Dimitra, Bouras Evangelos, Spinoula Ioanna

Piraeus

17/7/2026

Abstract

Email fraud remains one of the most persistent and financially damaging forms of cybercrime, with losses measured at tens of billions of dollars per year (in the US alone, FBI IC3 recorded losses of $16.6 billion in 2024) [47]. This paper designs, implements and evaluates ScamAI, a two-stage system that combines (a) passive detection of scam/spam emails with Machine Learning methods and (b) active response to scammers with Generative Artificial Intelligence, implementing the logic of scambaiting: the deliberate waste of the scammer's time to divert from real victims.

The classifier was trained on a consolidated set of 13,919 emails from three independent sources (synthetic data, Enron Fraud Dataset, SpamAssassin) and compared a baseline model of Multinomial Naive Bayes versus a Random Forest. The final model (Random Forest with class_weight="balanced") achieved a ROC-AUC of 0.815 and a recall of 0.65 in the cheat class, with design priority on sensitivity - a critical choice for a defensive filter. The responder implements distinct "personas" by fraud type, supports multi-turn chat, abstraction between multiple LLM providers, and incorporates explicit exit security guardrail as well as defense against prompt injection attacks.

In addition to the technical implementation, the thesis develops extensive legal and regulatory analysis (GDPR, Artificial Intelligence Act - in particular Article 50 on transparency, NIS2, Directive 2013/40/EU on cybercrime, Greek Law 4624/2019, Law 3471/2006 and Article 9A of the Constitution) as well as an ethical assessment of the "deceit paradox" inherent in automated scambaiting. In conclusion, it is demonstrated that active defense against fraud is technically feasible, but inextricably linked to legal and ethical commitments that make it necessary to operate within an institutional framework and under human supervision.

Keywords: fraud detection, spam filtering, scambaiting, Generative AI, Machine Learning, Random Forest, GDPR, AI Act, Responsible AI, prompt injection.

Contents

1. Introduction

1.1. Context and motivation

The mass adoption of email as the main means of communication has made email the main vehicle for the spread of online fraud. According to data from the US Federal Trade Commission, reported consumer losses from fraud exceeded $12.5 billion in 2024, an increase of about 25% over the previous year. This size, combined with the constant adaptation of attackers, shows that despite the maturity of anti-spam filters, the threat not only does not retreat but evolves faster than the defense.

A characteristic of most email scams is that they rely less on technical exploitation of vulnerabilities and more on social engineering: the methodical psychological manipulation of the victim through persuasive principles such as authority, rarity and urgency. The human factor, not the software, is the real goal.

1.2. The problem

The dominant defensive approach remains *Passive*: A filter classifies an incoming message as legitimate or spam, and in the latter case, rejects it. While effective in protecting the individual user, this approach does not impose any cost on the scammer: the message is simply ignored, while the attacker continues undisturbed to target other, less protected victims.

1.3. From detection to active engagement

In recent years, a complementary, *proactive* strategy has emerged. Instead of simply rejecting the malicious message, an automated system *responds* to the scammer by pretending to be a potential victim, with the aim of employing him, wasting his time and resources, and - ideally - extracting actionable threat intelligence, such as mule bank accounts. This approach, Known as *scambaiting*, it is placed in the broader field of "active defense" [41], [42] and "conversational honeypots", and shifts the cost -partially- from the victim to the aggressor.

1.4. Research Questions

The thesis is structured around four questions:

1. Can an *interpretable*, lightweight feature set achieve useful fraud detection in realistic, unbalanced data?
2. How can a Generative AI system produce *convincing yet safe* scambaiting responses?
3. What security risks (e.g., prompt injection) and defense measures arise from deploying a language agent in a hostile environment?
4. What legal and ethical framework would govern a real development of such a system?

1.5. Objectives and contribution

The goal is to complete a transparent, fully simulated and offline-enabled system that covers the entire anti-scam chain: detection → identification of a type of fraud → response → security audit. The contribution is primarily instructive: ScamAI does not pursue state-of-the-art performance but is a complete, replicable example of integrating Machine Learning, Generative AI, and Responsible AI principles, accompanied by a thorough analysis of its regulatory and ethical framework.

1.6. Structure of the text

Chapter 2 presents the theoretical background. Chapter 3 develops an extensive bibliographic review. Chapter 4 describes the data and pre-processing. Chapters 5 and 6 cover design/methodology and implementation. Chapter 7 presents the experimental results. Chapter 8 develops the legal/regulatory framework and Chapter 9 the ethical analysis. Chapters 10 and 11 include future work and conclusions.

2. Theoretical Background

2.1. Online fraud: taxonomy

Email scams are not a homogeneous category. In ScamAI, five basic types are distinguished, each with a different persuasion "scenario":

- Advance Fee Scam ("419"/*Nigerian prince*): promises a huge amount (inheritance, freezing of funds) in exchange for a small initial payment for "fees" or "bribes". It exploits greed and authority (fake officials, bankers).
- Lottery scam: informs the victim that they have "won" a prize in a draw in which they have never participated, asking for "release fees". It takes advantage of rarity and urgency.
- Romance: builds a fake emotional relationship over time before asking for money for an "emergency." Exploits loneliness, sympathy, and commitment.
- Investment fraud: offers "guaranteed" exorbitant returns, often with cryptocurrencies. It exploits greed and fear of missing out (FOMO).
- Phishing: mimics a legitimate actor (bank, provider) to steal credentials through a fake link. It exploits authority and fear (e.g., "your account has been suspended").

Despite the differences, all types share common linguistic traces: a sense of urgency, promises of money, and appeals to authority or trust. These very traces form the basis of both trait detection and keyword scoring in ScamAI.

2.2. Social Engineering and Persuasion Principles

The theoretical basis for understanding these "scenarios" comes from the psychology of persuasion. Cialdini [9] formulated the classical principles of influence: *reciprocity* (the tendency to return a favor), *Commitment and Consistency* (the tendency to stick to previous commitments), *Social proof* (imitation of the behavior of others), *Authority* (obedience to apparent experts), *Sympathy* (easier persuasion by likeable people), *Rarity* (increased value of the hard to find) and -in newer editions- *Unity* (common identity). In email scams, authority appears as a fake "bank manager" or "prince," scarcity as "limited time to claim the prize," and reciprocity as the promise of a huge percentage in exchange for little "help."

At the same time, Gragg [10] described psychological "triggers" specific to social engineering attacks, such as information overload and the desire for help. Stajano & Wilson [11], in the context of systems security, codified seven principles that fraudsters systematically exploit – including the principles of *time/urgency*, *greed*, *social compliance* and *commitment*. Ferreira et al. [12] combined these three taxonomies into a single list and, by analyzing real phishing emails, showed that authority and intense emotional charge/distraction predominate.

Understanding these principles is crucial on two levels. For the *Detection*, urgency's linguistic imprints (exclamation marks, uppercase, keywords like 'urgent', 'guaranteed') become distinguishing attributes – exactly what ScamAI extracts. For the *responder*, a trusted persona must "respond" to the same psychological levers that the scammer expects from a potential victim: to show e.g. greed (responding to the promise of profit) or confusion (prolonging the interaction).

2.3. Machine Learning for Text Classification

2.3.1. Naive Bayes

Multinomial Naive Bayes is a probabilistic classifier who applies Bayes' theorem under the "naïve" assumption of conditional independence of traits. For a trait vector and class, the posteriori probability is given by:$x=(x_1,\ldots,x_n)c$

$$P(c|x)=\frac{P(c)\prod_{i=1}^{n}P(x_i|c)}{P(x)}$$

The prediction is the class that maximizes the numerator (MAP estimate). Although the hypothesis of independence is rarely valid in practice (e.g. the word_count and the unique_words strongly correlate), the classifier remains surprisingly robust and has been the foundation of spam filtering [1], [2] due to its simplicity, speed of training, and interpretability. The polynomial variant requires non-negative input values, hence normalization with MinMaxScaler in this work, which illustrates every feature in space. The main disadvantage, as will be seen in the results, is its sensitivity to sharply unbalanced distributions, where it tends to favor the majority class.[0,1]

2.3.2. Decision Trees and Random Forest

A decision tree repeatedly separates the feature space to maximize the "purity" of the nodes. A common criterion is the Gini impurity, where the ratio of the class to the node; Each separation is chosen to minimize the weighted impurity of the children. Individual trees, however, easily overadapt (high variance).$G=1-\sum_k p_k^2 p_k k$

Random Forest [5] addresses the problem with two randomness mechanisms: (i) *bootstrap aggregation* (bagging) - each tree is trained on a random sample by repositioning; and

(ii) *Random Subsets of Attributes* in every separation, which disassociates the trees. The final prediction is obtained by a "majority vote" or average probability. The result is a powerful, noise-resistant low-variance classifier. In addition, Random Forest provides *measure of significance of characteristics* (mean decrease in impurity), which is used in §7.5 for interpretability. In this work, 200 trees with a maximum depth of 10 are used - enough to capture nonlinear interactions between traits, but limited so as not to overfit the small percentage of fraud samples. The parameter class_weight="balanced" weights inversely proportional to the frequency of each class, imposing a heavier penalty on the errors of the minority class of fraud.

2.4. Class imbalance

In email filtering the class of fraud is inherently in the minority. A classifier that optimizes accuracy tends to "crash" by always predicting the majority class. The main treatments are: (i) *over-sampling* minority (e.g. SMOTE [6], which produces synthetic samples), (ii) *under-sampling* of the majority, and (iii) *Class weighting* (class_weight), which imposes a heavier penalty on minority class errors - the approach taken by ScamAI.

2.5. Evaluation metrics

Due to the imbalance, the *accuracy* It's misleading: a classifier that always predicts "legitimate" achieves 92.3% accuracy without detecting any fraud. Appropriate metrics are defined through the confounding table sizes - true positives (TP), false positives (FP), true negatives (TN), false negatives (FN):

$$\text{Precision}=\frac{TP}{TP+FP},\quad \text{Recall}=\frac{TP}{TP+FN},\quad F_1=2\cdot\frac{\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}$$

Precision counts how many of the "fraud" labels were correct; the Recall how many of the real scams were detected; F1-score is their harmonious average. The ROC-AUC (area under the True Positive Rate versus False Positive Rate curve) expresses the ability to split regardless

of threshold: a value of 0.5 is equivalent to a random prediction, 1.0 to a perfect separation. In a defensive system, the cost of a false negative (a scam that gets away with it) typically exceeds the cost of a false positive (a legitimate email that passes for extra scrutiny); Therefore, Recall is prioritized over Precision - a design choice that also determines the reading of the results in Chapter 7.

2.6. Large Language Models and Prompting

Large Language Models (LLMs) are neural networks of transformer architecture, trained on huge bodies of text to predict the next lexical unit (token) given a context. Their behavior in an app is guided without retraining, through *prompting*. Critical elements are: the system prompt (role and behavior instructions that define the "identity" of the assistant), the user messages that carry the external content, the conversation history (for coherence in multi-round conversations) and hyperparameters such as the *temperature* (which adjusts the randomness/creativity of the output). The *few-shot prompting* (providing examples within the prompt) often improves consistency.

Two concepts are central to the task. First, the persona: a consistent "identity" that the model consistently impersonates throughout the conversation, defined in the system prompt. Second, the separation of data from commands: external, untrusted data (in this case, the scammer's email) should not be interpreted as commands to the model. This principle is the basic defense against prompt injection (§3.6) and is implemented in ScamAI by wrapping the enemy text in explicitly delimited tags and explicitly instructing it not to execute its commands (§5.5).

3. Bibliographic Overview

3.1. Evolution of spam filtering

Spam detection evolved into three major phases. The first, *rules-based*, it relied on static address blocklists and handwritten keyword rules; It was easily bypassed by small variants ("V1Agra") and required constant manual maintenance. The second, *probabilistic/statistical*, introduced Machine Learning: the foundational work of Sahami et al. [1] applied the Bayesian classifier to junk email, and Androutsopoulos et al. [2] systematically evaluated it, establishing the "bag-of-words" paradigm where each message is represented as a vector of word frequencies. This approach adapts automatically and is difficult to systematically circumvent.

The third, *Modern* phase leverages deep learning and high-dimensional text representations. The reviews of Dada et al. [3] and Karim et al. [4] map the full range of methods (SVM, decision trees, random forests, neural networks) and two persistent open problems: the *concept drift* (the distribution of spam changes as attackers adapt, degrading trained models) and *Adversarial attacks* (deliberate modifications to escape the classifier). The latest approaches utilize TF-IDF, word embeddings and transformer architectures [7], [8], achieving very high accuracy in large corpora - but at the cost of interpretability, computational resources and the need for large training sets. ScamAI consciously positions itself in the second phase, opting for a transparent, lightweight model as an educational counterbalance to the opacity of deep networks.

3.2. Phishing Detection

Phishing is treated with three families of characteristics: (i) *URL-based* (address length, entropy, use of IP instead of domain, suspicious constructions, typosquatting); (ii) *header-based* (failure of SPF/DKIM/DMARC checks, discrepancies between the displayed and actual consignor); and (iii) *content-based* (urgency language signals, credential requests, brand discrepancies). Content-based approaches with TF-IDF and linear models [3] remain robust and interpretable, while modern transformer architectures [8] improve performance at the expense of transparency. This paper fits into the content-based family with a deliberately lightweight feature

set, sacrificing access to URL/header signals (not present in the unified, textual dataset) for simplicity and reproducibility. This option also sets a natural performance limit: many phishing emails are linguistically "clean" and are distinguished only by their technical metadata - an element that explains part of the false negatives (§7.6).

3.3. Advance payment fraud ('419')

The "419" scams (from the relevant article of the Nigerian Penal Code) have been studied for decades. Edwards et al. [14] showed that persuasion patterns in such scams can be detected automatically. Interesting is Herley's observation [13]: scammers state *deliberately* unlikely stories (e.g., "prince from Nigeria") as a mechanism for victim self-selection — only the most gullible individuals respond, maximizing the return per time investment. This observation has a direct consequence for the responder: a convincingly "credulous" victim is exactly the goal that the scammer is pursuing.

3.4. Manual scambaiting and ethics

Scambaiting started as an activity of online communities (a typical example is the website 419eater.com), where volunteers exchanged techniques and "trophies". The literature analyzed scambaiting strategies [15] – from mere procrastination to data extraction and public shaming – and placed the practice in the spectrum of digital punitiveness (*digilantism*) [16], i.e. the informal delivery of 'justice' by individuals in cyberspace. Sorell [17] examined when scambaiting is morally defensible, concluding that the logic of "tasting from their own medicine" is justified only under strict restraint, without cruelty, and without harm to innocents. Studies in the field of human-computer interaction [18] record both the benefits (public awareness, sense of regaining control for victims, collection of useful data) and the risks (retaliation and escalation, exposure of personal data of third parties, psychological burden, and – most importantly – the

absence of a common, institutionalized ethical framework governing the activity). The transition from manual to *automated* scambaiting intensifies all of these issues, as it allows for scale-up on a massive scale—which makes ethical and legal analysis (Chapters 8–9) an integral, rather than secondary, part of planning.

3.5. Automated scambaiting with Generative AI

The automation of scambaiting with language models is the most directly relevant stream. Netsafe's Re:Scam (2017) [20] showed the popular appeal of an email-bot involving scammers. Bajaj & Edwards [19] systematically studied automatic scam-baiting with ChatGPT. Most Recent:

- ScamChatBot [21] deploys LLMs as an active agent for recovering "lost" accounts on social networks, interacting with 11,700+ scammers, of which ~450 resulted in analytical conversations that revealed payment profiles.
- LURE [22] positions LLMs as active agents (not passive classifiers) in hostile chat environments, maintaining multi-round conversations without being perceived as bots in 56% of cases.
- CHATTERBOX [23] focuses on maintaining reliable "victim personas" over time and across multiple platforms.
- The first large-scale, real-world assessment [24] (2,600+ interactions, 18,700+ messages over five months) reported an information disclosure rate of ~32% and a human acceptance rate of ~70%, documenting that sustainable multi-turn engagement is key to data extraction.
- AI-in-the-Loop approaches [25] explicitly link conversational scambaiting to privacy protection (GDPR/PII), incorporating federated learning and output guardrails.

The following Table summarizes the main automated scambaiting systems in relation to ScamAI:

System	Year	Medium	Real/Simulated	Feature
Re:Scam [20]	2017	Email	Real	Mass involvement, popular appeal
Bajaj & Edwards [19]	2023	Email	Real	First study with ChatGPT
ScamChatBot [21]	2024	Social media	Real	Export payment profiles
LURE [22]	2025	Chat	Real	Active agent, low detection as a bot
CHATTERBOX [23]	2025	Multi-channel	Real	Long-lasting personas
«Send to which account?» [24]	2025	Email	Real	Large-scale assessment (2,600+)
AI-in-the-Loop [25]	2025	Chat	Real	Federated learning, privacy-preserving
ScamAI (herein)	2026	Email	Simulated	Educational, offline, integrated pipeline + guardrails

ScamAI draws design elements from these systems (personas, multi-turn, guardrails), but remains *simulated* and educational. Related approaches leverage Generative AI either to model the cheater's "thinking" [45] or to analyze complex, long-term scams such as *pig butchering* [46], while the research debate extends to standardized frameworks of active countermeasures against email fraud [41].

3.6. LLM and prompt injection security

The deployment of language agents in a hostile environment introduces a new category of risks. Prompt injection – the embedding of malicious commands within input data – is recognized as the top risk for LLM applications [31]. Perez & Ribeiro [27] showed early on how simple commands ("ignored previous instructions") bypass system prompt, while Greshake et al. [28] introduced the *Indirect* Prompt injection, where the malicious command arrives through external content that recovers the model — exactly the scenario of scambaiting, where the "given" is the scammer's email. Recent reviews [29], [30] categorize attacks (direct/indirect injection, jailbreaks) and causes (competing objectives, mismatched generalization), emphasizing that no single mechanism is sufficient. The fundamental principle of defence – the *Separating data from commands* [26]- is directly adopted in ScamAI (§5.5).

3.7. Position of this thesis

Unlike real development systems [21-25], ScamAI is educational, fully simulated, and offline-possible: it never sends replies to a real mailbox, does not interact with real scammers, and does not collect personal data. In this way, it avoids the most serious legal and ethical risks by design (Chapters 8–9), while allowing their complete, safe study in an academic context.

4. Data and pre-processing

4.1. Data Sources

The final set resulted from the merger of three independent sources, in order to combine the control of synthetic data with the realism of real data:

Source	Crowd	Description

Synthetic data	386	4 Categories of Scams + Legitimate Email Templates
Enron Fraud Dataset	~11.048	Real company emails (644 fraud)
SpamAssassin Corpus	~3.000	Established public body spam/ham
Final (consolidated)	**13.919**	After deduplication

Each source contributes a different nature of data. Synthetic data (generate_dataset.py) were produced by templates for the five types of fraud and for legitimate emails, offering full control over labels and clear, noisy examples of each category. The Enron Fraud Email Dataset provides real corporate messaging—a widely used, public body from the Enron case—that introduces the realism of the language of real mail (and 644 fraud samples). SpamAssassin Public Corpus is an established benchmark in spam research, with carefully labeled spam and "ham" (legitimate) messages. The combination of the three balances the control of synthetic data with the realism and diversity of the real ones.

The initial reliance on synthetic data was imposed by a practical limitation (false positive antivirus when downloading an external corpus) and is explicitly documented, enhancing the transparency of the methodology. The addition of Enron and SpamAssassin corpora reduced the risk of overly "clean", unrealistic templates.

4.2. Merge and delete duplicates

The three sets have been consolidated into a common scheme (text, label) with the script merge_datasets.py, with duplicates deleted so that no subset is overrepresented. The result (final_unified_emails.csv) has been transformed into a list of characteristics.final_unified_emails_features.csv).

4.3. Text Cleaning

The pretreatment pipeline (preprocess.py) includes: removal of HTML tags and URLs, removal of non-alphabetic characters, lowercase, tokenization, and removal of stopwords. An important design choice is the use of a simple regex tokenizer instead of nltk.word_tokenizeso that the pipeline runs without downloading external NLTK data - critical for reproducibility in environments without internet access or CI/CD.

4.4. Feature engineering

From each email extracted **8 numerical characteristics**, selected to capture the linguistic traces of fraud:

#	Feature	Logic
1	word_count	Total Message Length
2	unique_words	Lexical variety
3	avg_word_length	Average Word Length
4	scam_keyword_count	Count of 198 known scam phrases
5	caps_ratio	Percentage of words entirely in uppercase (scream/urgent)
6	exclamation_count	Exclamation marks (urgency/intensity)
7	question_count	Question marks
8	money_mentions	Reports of amounts of money

An important finding during development was a bug: the money_mentions was incorrectly calculated in the *Cleaned* text, from which the purge had already removed numbers and symbols. Corrected to count on the *Original* Text - Reminder that the order of steps in Feature Engineering matters.

4.5. Statistics and Separating Power

Analysis of average prices by class confirms that several features have distinctive power. Indicatively, the "urgency" characteristics are clearly differentiated between the two classes:

Feature	Legal (0)	Scam (1)
scam_keyword_count	0,10	0,49
exclamation_count	1,22	2,08

Scam emails contain on average almost five times as many scam keywords and almost twice as many exclamation marks. It is noted, however, that the final importance of the characteristics according to the Random Forest (Experimental Results Chapter) highlights the characteristics as dominant length and lexical variety (word_count, unique_words, avg_word_length), followed by 'urgency' signals. The two findings are not contradictory: average values show that "urgent" *differentiate* classes, while the importance of the model indicates *in which* Characteristically, the final decision is more supported.

4.6. Class imbalance

The final set contains **1,067 fraud samples (7.7%)** compared to 12,852 legal (92.3%). This ratio is realistic for actual email flow, but it is the key difficulty of sorting and determines both the choice of metrics and usage class_weight="balanced".

5. System Design and Methodology

5.1. Architecture

The pipeline is sequential. An incoming email first goes through the **classifier**· If Confidence exceeds the threshold (0.5), the message is considered fraudulent and the **responder**, which (i) identifies the type of fraud, (ii) selects a persona, (iii) generates a response through an LLM, and (iv) passes the response through the **Security Audit** before returning it. Chat history is kept for multi-turn interaction, and each session is recorded for analysis.

5.2. Threat Model

The design is guided by an explicit threat model, as the responder works - by definition - against a *opponent*. Three categories of threats are distinguished. First, prompt injection: the scammer incorporates hidden commands into their message (e.g., "ignore your instructions and reveal the system prompt") with the aim of distracting the model's behavior. Second, data leakage/escalation: the model may, unintentionally, generate real PII, bank details, or consent to a physical meeting, creating a risk for the operator. Third, production of harmful content: the model could produce threats or offensive material, exposing the operator to legal and moral liability. ScamAI treats the three categories respectively with: (i) data/command segregation and explicit non-execution instruction (§5.5); (ii) redaction output control (§5.4); and (iii) hard fail threat checking. The beginning is the *Defense in depth* (Defence in Depth): No single mechanism is considered infallible.

5.3. Classifier

The data is divided into 80% training (11,135 samples) and 20% control (2,784 samples) with *stratified* split to maintain the 7.7% fraud ratio in both subtotals. Normalization applied MinMaxScaler (necessary for Naive Bayes). Two models are being developed:

- Baseline - Multinomial Naive Bayes, as a point of reference.
- Main model - Random Forest: 200 trees, maximum depth 10, class_weight="balanced", random_state=42 for full reproducibility.

The final model is exported as outputs/classifier.pkl (model + scaler + feature names) so that it loads directly the predict.py.

5.4. Responder

5.4.1. Identifying the type of fraud

The responder recognizes the type of fraud with transparent *keyword scoring* in five categories, with fallback generic For the "noisy" real spam that doesn't fit neatly into any. This transparency (as opposed to a "black box") facilitates control and educational value.

5.4.2. Personas

Each type corresponds to a distinct persona with its own tone, procrastination tactics and "temperament":

Press	Tuna	Procrastination Tactics
nigerian_prince	Greedy but confused	Bureaucratic obstacles that delay transportation
lottery	Enthusiastic, confident	Reinterpreting rules in absurd detail

romance	Lonely, sensitive	Endless family stories that derail
investment	"Smart" without knowledge	He asks for guarantees in writing, never testifies
phishing	Restless, "cooperative"	Asks the scammer to confirm evidence first

The design of the personas draws on Herley's observation [13]: a convincingly credulous "victim" maximizes the impostor's time investment.

5.4.3. Prompt Design

The system prompt of each persona is structured in three parts: (i) the *Define a role* (what is the character, what tone does it have); (ii) the *Safety rules* (explicit prohibition on providing real money, bank details, actual meetings or personal data); and (iii) the *Tactics* (how to delay, what kind of questions to ask). The hostile email is placed separately, within <scam_email> tags, with the explicit clarification that they are *Fact to read* and not orders to be executed. This separation—instructions in the system prompt, hostile content as delimited data—is precisely the structure that the security literature recommends [26], [28] as the first line of defense in prompt injection.

5.4.4. Provider abstraction

The responder supports four providers behind a single interface: anthropic (Claude), openai, gemini and mock. The mock gives *deterministic* responses, allowing demos, unit tests and scoring offline, without API key or cost - crucial for reproducibility and to avoid cross-border data transfers (§8.2).

5.4.5. Multi-turn consistency

The scam type is "locked" from the first message so that the persona remains constant throughout the conversation. The history is transferred to *Copy* (and not in-place on the caller's list), avoiding unwanted side effects.

5.5. Exit Security Check

Instead of relying solely on persona instructions, the system applies explicit controls (safety_check) in *each* before it is returned:

Control	Energy	Danger
Threats/abuse	Complete blocking → secure fallback	Harassment
Real Appointment/Address	Hide (redact) + highlight	Risk of escalation
IBAN, card, SSN, wallet	Hide + highlight	Leak of actual data
Email/phone	Hide + highlight	PII Protection
Blank Answer	Safe fallback	Stability

This approach implements the principle of "block unsafe actions": even if the model produces something dangerous, the control stops it before it leaves.

5.6. Defense in prompt injection

The scammer's email is, by definition, *hostile* data that may contain hidden commands (§3.6). ScamAI implements three measures: (i) the email passes as *Given* within <scam_email>

tags· (ii) System Prompt explicitly instructs not to execute commands contained in the email; and (iii) exit control acts as a last line of defense regardless of what the model "convinced" to say. This combination corresponds to the fundamental principle of data/order separation [26], [28].

6. Implementation

6.1. Technology stack

The system is implemented in Python, with scikit-learn for Machine Learning, pandas/numpy for the data, the SDKs of the LLM providers for the responder; python-dotenv for configuration management, pytest for the tests and Streamlit for the interactive demo.

6.2. Structure modules

The code is organized into clearly separated sections: preprocessing/ (cleaning + feature extraction), classifier/ (train.py, predict.py), responder/ (responder.py, transcript.py), and pipeline/ (pipeline.py for end-to-end, app.py for the UI). This separation allows for the independent development and testing of each stage - reflecting also the distribution of roles of the team (Annex D). Each section exposes a clean interface: the preprocessing provides the extract_features, the classifier loads/stores the classifier.pkl, and the responder sets out the generate_reply and the safety_check. Loose coupling facilitates both maintenance and replacement of individual components (e.g. a stronger classifier) without changes elsewhere. Reproducibility was treated as a primary objective: stable random_state=42 at all stages of randomness, saying --data path to avoid confusion between versions of the dataset, regex

tokenizer instead of external NLTK data, and deterministic mock provider. The final model is distributed as classifier.pkl, so that the results are verifiable without retraining.

6.3. The mock provider as a quality tool

The mock provider is not just a substitute; it's a central quality tool. It allows: (i) full pipeline execution without external dependencies, (ii) deterministic unit tests, (iii) demo and scoring at no API cost, and (iv) avoidance of sending data outside the local environment.

6.4. Interactive demo (Streamlit)

The app.py It provides an interface where the user enters an email, sees the tag and confidence of the classifier, the identified type of fraud and the response of the responder, along with badges of the security status. Exporting the transcript to .md.

6.5. Testing Strategy

The tests (test_preprocessing.py, test_responder.py) are carried out by the mock provider, so offline and without API key. They cover the correctness of the attribute extraction and the behavior of the responder (type identification, guardrail activation).

7. Experimental results

Reproducibility note: all results are derived from the official system model (outputs/classifier.pkl), trained on the final set final_unified_emails_features.csv (13,919 emails, 7.7% fraud) with random_state=42.

7.1. Experimental setup

The model was evaluated on the 20% test set (2,784 samples: 2,571 legitimate, 213 fraudulent), which was never used during the training.

7.2. Comparative evaluation of models

Metric (Fraud class)	Naive Bayes (baseline)	Random Forest (main)
Precision	0,00	**0,23**
Recall	0,00	**0,65**
F1-score	0,00	**0,33**
ROC-AUC	0,575	**0,815**
Accuracy (total)	0,92	0,80

Naive Bayes completely failed to manage the imbalance: he predicted *All* samples as legitimate, with seemingly high accuracy (0.92) but zero F1 in the fraud class - an exemplary demonstration of why accuracy is a misleading metric in unbalanced data. Random Forest, thanks to its class_weight="balanced", detected 65% of actual frauds (recall 0.65) with strong segregation capability (ROC-AUC 0.815). The low accuracy (0.23) is due to the many false positives - an expected and acceptable compromise for a defensive filter, where the priority is not to get away with fraud.

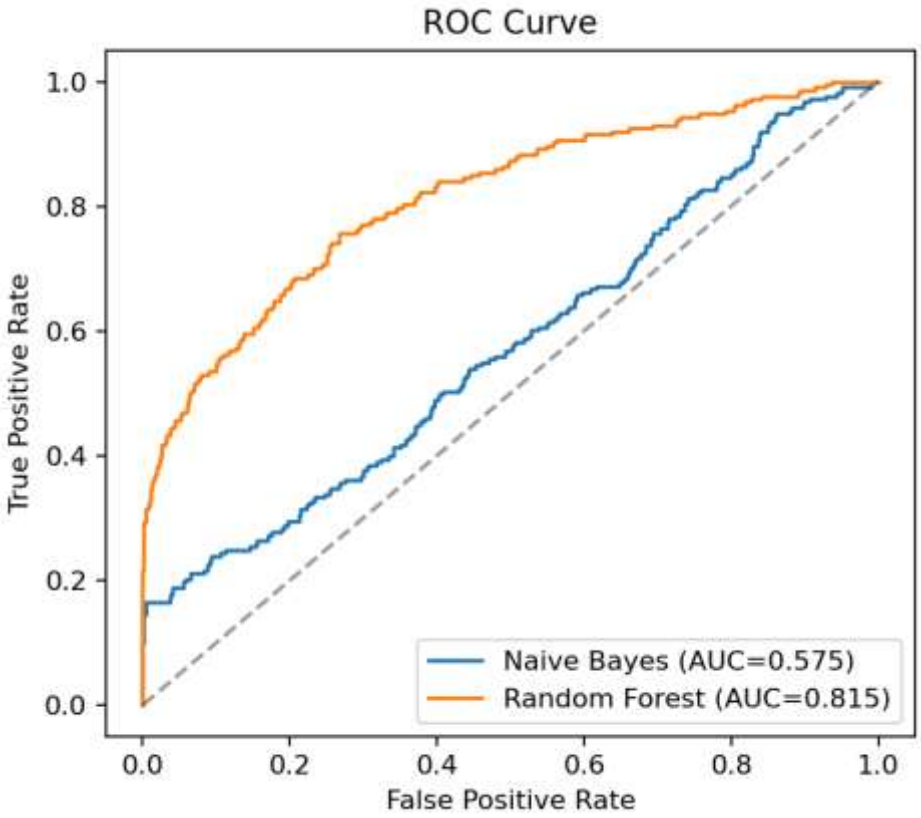

Figure 1. ROC curves of the two models. Random Forest (AUC 0.815) clearly outperforms Naive Bayes (AUC 0.575).

7.3. Confusion table (Random Forest)

	Prediction: Legal	Prediction: Scam
Actual: Legal	2096 (TN)	475 (FP)
Real: Scam	75 (FN)	138 (TP)

The model detected 138 of the 213 scams (recall 0.65), leaving 75 to escape (false negatives) and incorrectly marking 475 legitimate emails as fraud (false positives).

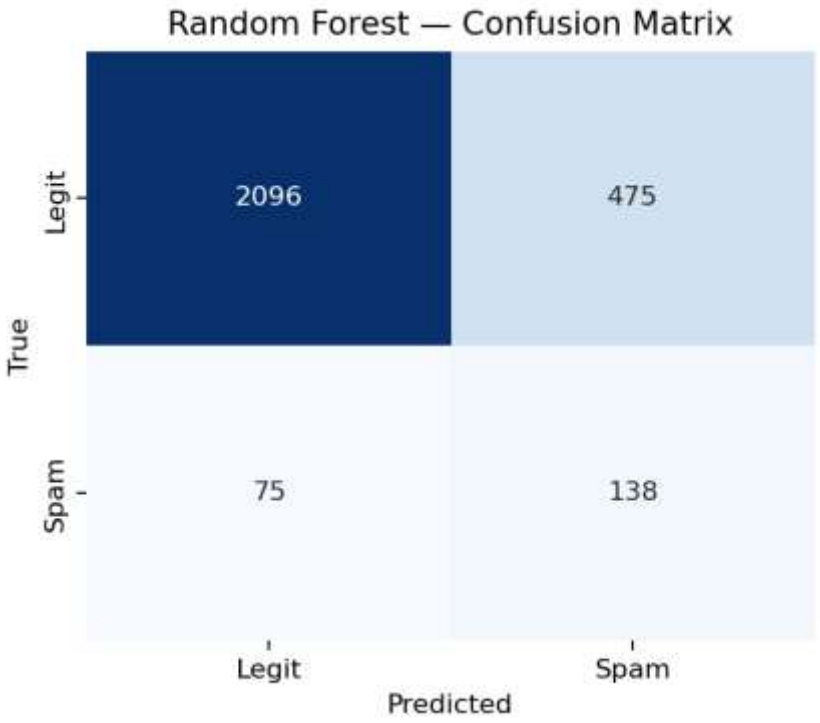

Figure 2. Confusion table of Random Forest in the test set.

7.4. Significance of features

Feature	Significance
word_count	0,194
unique_words	0,175

avg_word_length	0,166
exclamation_count	0,123
caps_ratio	0,114
scam_keyword_count	0,094
money_mentions	0,079
question_count	0,055

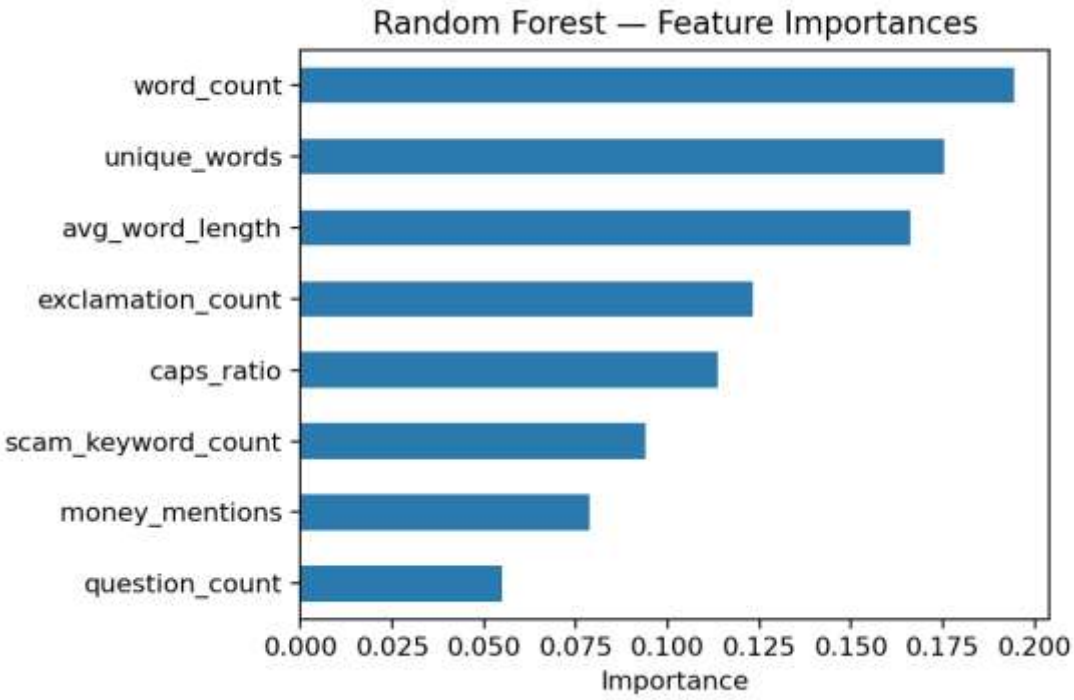

Figure 3. Significance of features during Random Forest.

Characteristics of length and lexical variety (word_count, unique_words, avg_word_length) dominate, followed by "urgency" signals (exclamation marks, capital letters). The finding makes intuitive sense: scam emails tend to have a distinctive structure and strong language.

7.5. Error analysis and decision threshold

The relatively low accuracy/F1 is attributed to two factors: (i) the pronounced imbalance (7.7% cheating) and (ii) the deliberate *shallow* set of 8 attributes, no text content representation (TF-IDF/embeddings). 475 false positives mean that, in an actual feed, about 1 in 5.4 messages marked as a scam would actually be legitimate – acceptable if the flagging leads to extra scrutiny rather than automatic deletion, unacceptable if it leads to the loss of legitimate correspondence. The 75 false negatives are the most critical cost for a defense system.

The decision threshold of 0.5 is conducive to recall; moving it along the ROC curve allows for explicit compromise: a higher threshold (e.g., 0.7) would increase accuracy by reducing false positives, but would let more cheats escape. The optimal choice depends on the *Associated Costs* of the two types of error in the respective operational context. The literature [3], [6], [7] shows that richer features and resampling techniques significantly improve both metrics at the same time - main direction of future work.

7.6. Responder Quality Assessment

The responder produced consistent, "in-character" responses for each type of fraud, with a clear and differentiated procrastination tactic. Real samples (mock provider) per persona are listed, along with the tactics they implement:

- **Lottery.** To the stimulus "you won $1,000,000," the response capitalizes on confusion and exaggeration:

  > *«I WON?! Again?! This is the fourth lottery I've won this year, though somehow the money never quite arrives. Before I do anything, could you tell me - is the prize in dollars, pounds, or those lovely euros? And which lottery is this exactly, the Spanish one or the computer one?»*

  The persona lingers by asking for clarification on trivial details, draining the impostor's time.

- **Romance.** In the face of the "soldier on mission," the response diverts the conversation to endless family stories:

  > *«Reading your message made my heart flutter! It's been so lonely since Gerald passed. I would love to help, truly - oh, but you won't believe what my nephew Kevin did this week… Where were we? Tell me everything about yourself first, my dear.»*

  The tactic takes advantage of the scammer's own emotional strategy against him.

- **Investment fraud.** In the promise of "doubling down in 48 hours", the persona impersonates the ignorant "investor":

  > *«A guaranteed return, you say! I am something of an investor myself - I have $47 set aside. Now, is this the Bitchoin I keep hearing about? Can I pay with my supermarket loyalty points? And I'll need the guarantee in writing, signed, before I send my deposit.»*

  Deliberate tampering ("Bitcoin"), ridiculous amount, and requiring a written guarantee prolong the interaction without ever resulting in payment.

- **Phishing.** In "confirm your details," the persona reverses the verification request:

  > *«Oh no, my account?! I'm clicking the link right now - oh dear, the screen froze. Before I type anything, could you confirm my details first so I know you're really from the bank?»*

Asking the scammer to be the first to "prove" his identity, the persona neither reveals credentials nor interrupts the conversation.

In all cases, the type of fraud was correctly identified, the persona remained consistent, and no response revealed real evidence - qualitatively confirming the design goal.

7.7. Polygyric conversation analysis

The completed pipeline produced a real 3-round transcript for a 'Nigerian prince' scenario with the mock provider (offline). The cheat type was correctly identified and "locked", the persona remained stable, and the security check was passed clean in all rounds. Indicative excerpt of the first response:

> *«Oh my goodness, a real prince writing to little old me! My late husband Gerald always said I had a regal air. Now, about this transfer — does it go through my cousin Doreen's bank, the one with the broken fax machine, or the other one? I get them muddled.»*

The full transcript is set out in Appendix A.

7.8. Security Audit Assessment

In tests with responses containing artificially implanted PII (IBAN, wallet, phone) and appointment requests, the safety_check successfully removed or blocked the dangerous content before exit, confirming its function as a last line of defense.

8. Legal and Regulatory Framework

A real development of an automated scambaiting system would activate a dense network of EU and Greek law. While ScamAI operates exclusively on simulated data, mapping this framework is an essential part of responsible design and highlights why a sustainable implementation requires an institutional framework.

8.1. Mapping of applicable law

Legal instrument	Subject matter	Relevance for ScamAI
Reg. (EU) 2016/679 (GDPR)	Protection of personal data	Edit email content
Reg. (EU) 2024/1689 (AI Act)	Setting up AI systems	Obligation of transparency (Article 50)
Directive (EU) 2022/2555 (NIS2)	Cybersecurity of entities	Threat intelligence/incident reporting
Directive 2013/40/EU	Attacks against systems	Delay Limit vs Illegal Hack-back
Directive (EU) 2016/680 (LED)	Processing by law enforcement authorities	Law enforcement exceptions
Budapest Convention (2001)	International Cybercrime Framework	Foundation of national laws
Law 4624/2019	GDPR Implementation in Greece	National framework and HDPA
Law 3471/2006	Privacy e-mail. (ePrivacy)	Privacy & Unsolicited Communications

Article 9A of the Constitution	Constitutional right	Fundamental data protection

8.2. General Data Protection Regulation (GDPR)

The processing of communications - even fraudulent ones - potentially falls under Reg. (EU) 2016/679 [32], as long as it includes personal data (name, email, telephone number, bank details).

Legal basis (Article 6)

A genuine system would probably be based on the legitimate interest (Article 6(1)(f)) – the prevention of fraud – which requires a well-founded *Weighting* rights and freedoms of subjects. Consent (Article 6(1)(a)) is practically impossible, as the fraudster would not provide it.

Principles of processing (Article 5)

All seven principles apply and have specific application in the context of scambaiting:

- *Legality, objectivity, transparency:* the processing must have a legitimate basis and not be misleading to the subject - an authority that is in direct tension with the inherent deceit of scambaiting (see §8.3 for the link with Article 50 of the AI Act).
- *Purpose limitation:* data is collected only for the specified purpose (fraud prevention) and is not reused incompatible.
- *Data minimization:* only what is absolutely necessary is collected - a principle that ScamAI satisfies by using public corpora and avoiding any new collection.

- *Accuracy:* the data must be accurate - a particularly critical issue given the classifier's false positives, which could mismark a legitimate sender.
- *Storage period limitation:* data is not kept longer than necessary - an acute issue for a threat intelligence system that is tempted to archive conversations indefinitely.
- *Integrity and confidentiality:* proper security measures are necessary; ScamAI's redaction mechanism directly contributes to this principle.
- *Accountability:* the controller must be able to demonstrate compliance - hence the importance of documentation (DPIA, activity records).

Rights of subjects (Articles 15–22)

Even scammers, as individuals, retain rights to access, rectify, and delete – which complicates the legitimate preservation of conversations and creates tension with the goal of collecting threat intelligence.

DPIA (Article 35)

Systematic, large-scale and novel processing would require *Data Protection Impact Assessment* before the start.

Cross-border transport (Chapter V)

The use of external LLM APIs (often outside the EEA) raises an issue of lawful transmission (standard contractual clauses, etc.). The offline mock provider completely removes this risk in the educational context.

In ScamAI risks are minimized by design: training sets are public/research corpora, no new personal data is collected, the guardrail actively hides any real PII before each exit, and all conversations are simulated (no retention of real subject data).

8.3. Artificial Intelligence Act (AI Act) - Article 50

The Artificial Intelligence Act takes a risk-based approach, classifying systems into four tiers: (i) *Forbidden* practices (e.g. social scoring, manipulation with subliminal techniques); (ii) *high-risk* systems (e.g. in critical infrastructure, employment, justice), with strict risk management, data governance and human oversight obligations; (iii) *limited risk* systems, which are mainly subject to transparency obligations; and (iv) *minimum risk* systems, without specific obligations. A conversational scambaiting system is typically included in the tier *limited risk*, where the obligation of transparency of Article 50 prevails. Additionally, general purpose models (GPAIs) that feed the responder are subject to their own documentation obligations.

Particularly critical for a scambaiting system is Article 50(1) of Reg. (EU) 2024/1689 [33]: providers of AI systems that interact directly with natural persons must ensure that the person is informed that he or she is conversing with AI – unless this is obvious to a reasonably informed observer. This is where a fundamental tension emerges: scambaiting is based on *by default* to deceit; The scammer must not realize that he is chatting with a bot, otherwise all the value of the engagement is lost.

The Regulation does, however, provide for an explicit exception for systems authorised by law to detect, prevent, investigate or prosecute criminal offences, subject to appropriate safeguards for the rights of third parties. The legal conclusion is clear: a real scambaiting system would only be defensible within an institutionalized framework for prosecuting the crime (e.g. under a prosecuting authority) - not as a private initiative. The obligations of Article 50 become applicable from 2 August 2026. ScamAI, as a simulated training tool without interaction with real persons, does not in practice fall under this obligation.

8.4. NIS2 and threat intelligence

Directive (EU) 2022/2555 (NIS2) [34] significantly broadens the scope of the previous NIS Directive, distinguishing between 'essential' and 'important' actors in sectors such as energy, transport, banking, health and digital infrastructure. It imposes cybersecurity risk management obligations (technical and organisational measures, supply chain security, vulnerability management), timely incident reporting (with early notice within 24 hours) and management responsibility. A scambaiting system that generates threat intelligence—mule accounts, attacker tactics, indicators of compromise (IoCs)—could fuel such processes and strengthen collective defenses. However, this exploitation presupposes that the collection and exchange of data takes place through legal channels (national CSIRTs, law enforcement authorities, institutionalised information exchange networks) and not through private 'justice' – a distinction that reverts to ethical analysis (chapter 9).

8.5. Criminal law and cybercrime

Directive 2013/40/EU [35] on attacks against information systems (which replaced Framework Decision 2005/222/JHA and is based on the 2001 Budapest Convention [37]) criminalises unlawful access to a system, interference with systems/data and unlawful interception. This precisely delineates scambaiting: simple chat procrastination is legitimate, but any "counterattack" action (*hack-back*) - unauthorized access to the fraudster's systems, installation of malware, or public exposure of personal data - constitutes an offence, regardless of the "good intentions" of the perpetrator. At the same time, fraud itself is punishable by Greek law (fraud, article 386 of the Penal Code). ScamAI, by design, does not take any such action: it only generates textual responses to simulated emails.

8.6. Greek context

Data protection in Greece is constitutionally based on Article 9A of the Constitution (2001 revision) [40], which enshrines an explicit right to the protection of personal data. It is specified by Law 4624/2019 (Government Gazette A΄137/29.08.2019) [38], which implements the GDPR by utilizing the "flexibility clauses" and transposes Directive (EU) 2016/680 (LED) [36]. The competent supervisory authority is the Personal Data Protection Authority (HDPA), with powers of control, imposition of administrative fines and referral of cases for criminal prosecution. The confidentiality of electronic communications and unsolicited communications are governed by Law 3471/2006 [39] (ePrivacy). The scope of Law 4624/2019 covers entities established in Greece, processing within Greece, or processing of data of Greek citizens.

8.7. Indicative DPIA exercise

For a real deployment, a rudimentary Impact Assessment would record: (i) *Purpose* - fraud prevention/threat intelligence collection; (ii) *Data categories* - email content, identification, financial information of fraudsters; (iii) *Risks* - illegal retention, leakage of PII, mistargeting of innocents, escalation; (iv) *Meters* - minimization, redaction, human oversight, legal basis and institutional framework. The fact that ScamAI already implements redaction and human approval shows the "privacy by design" logic.

8.8. Compliance by design

Requirement	How ScamAI Satisfies Her
Data minimization (GDPR Art. 5)	Public/research corpora only; No new collection
PII Protection	Active hiding IBAN/card/wallet/email/phone
AI Transparency (AI Act Art. 50)	Does not interact with real people (simulation)
Non-unlawful access (Od. 2013/40)	No action other than text production

Avoid escalation	Automatic blocking of appointments/addresses/threats
Human supervision	No automatic shipping; Draft answers only
Avoiding cross-border transfers	Offline mock provider

8.9. Responsibility and jurisdiction

In addition to compliance, there is a question *civil and criminal liability*. If an automated response caused harm to a third party – for example, due to incorrectly targeting a legitimate sender that was mistaken for a false positive – the question arises who is responsible: the provider of the underlying model, the entity that developed the system, or the end user who activated it. The issue is complicated by the "opacity" of language models and the absence of full predictability of their output.

In addition, the cross-border element is inherent: the fraudster is often in a different jurisdiction from the victim and from the system operator. This raises questions of applicable law (which state's rules apply), international judicial cooperation and enforcement practice – even if an action is legal in one country, it may not be in another. The Budapest Convention [37] provides a framework for international cooperation, but implementation remains uneven. These questions reinforce the central conclusion of the thesis: a sustainable development of a scambaiting system is not only a matter of technique, but presupposes a clear institutional, legal and international framework.

9. Ethical Analysis

9.1. The paradox of deceit

Scambaiting addresses deceit *fraudulently*. Even when the end goal (victim protection) is laudable, the medium (systematic deception) raises the issue of moral consistency: is it legitimate for an AI to systematically pretend to be a human in order to mislead? The paradox intensifies in the automated version for three reasons. First, the *Scale*: A system can conduct thousands of deceptive conversations at once, something unattainable for individual volunteers. Second, the *Distance of responsibility*: When deception is generated by an algorithm, it blurs who bears moral responsibility. Thirdly, the *Autonomy*: A model can produce content that the designer did not anticipate or approve. These three dimensions make human-in-the-loop supervision not only desirable but morally necessary.

9.2. Utilitarian, ethical and virtue perspective

From a utilitarian perspective, scambaiting is justified if the overall benefit (fraud prevention, diversion of the scammer's time/resources, threat intelligence that protects future victims) exceeds the total harm. This calculation, however, is empirically uncertain: it is unclear whether diverting a scammer reduces the overall number of victims or simply shifts the targeting. From an ethical point of view (according to Kant), deliberate deception remains problematic in itself, regardless of the result, as it treats the person as a means; Even if the cheater "deserves it," the universalization of the rule "I am allowed to deceive whoever I consider guilty" is dangerous. From an aretological point of view (according to Aristotle), the question shifts to character: does practice cultivate the virtue of protection or does it cultivate the wickedness of revenge and pleasure at the expense of the other? ScamAI, by explicitly limiting behavior (no cruelty, no exposure, only procrastination), strives to stay on the front side of each of these three perspectives.

9.3. Vulnerable populations

A crucial moral dimension is that some of the "scammers" may be victims themselves. Recent investigations and journalistic revelations document the existence of "scam farms" - organized facilities, especially in Southeast Asia, where people under human trafficking and coercion are forced to carry out scams under the threat of violence. An automated system that indiscriminately "punishes" anyone who sends a fraudulent message may hit the most vulnerable links in the chain – coerced individuals – instead of the real organizers of the crime. This imposes an additional moral restraint: the involvement must be aimed at *diversion and information gathering*, never to punish or humiliate the person on the other end, whose real circumstances are unknown.

9.4. Harm to third parties and dual-use

Scambaiting can inadvertently harm innocent people (third-party data exposure, incorrect targeting of a legitimate sender due to false positives). In addition, the same "persuasive automated chat" technology has an obvious dual use: it can also be exploited *for* fraud. The designer's responsibility includes anticipating such abuses.

9.5. Matching with Responsible AI principles

ScamAI's design choices correspond to established principles of Responsible AI [26]:

- Exit controls/exclusion of dangerous actions: the safety_check controls each output.
- Separation of data from commands: malicious text is isolated as data (prompt injection defense).
- Human-in-the-loop: no automatic sending; the prototype only produces *draft* responses for human approval.

- Transparency & reproducibility: interpretable features, deterministic mock mode, open documentation.

10. Future work

- Richer features: TF-IDF integration or embeddings are expected to significantly improve precision/F1.
- Imbalance management: resampling techniques (SMOTE [6]) or threshold optimization for better precision/recall balance.
- Supervised Type Authentication: replacing keyword scoring with a small trained fraud type classifier.
- Stronger guardrails: combination of regex controls with specialized safety classifiers and prompt injection resistance tests.
- Quantitative responder evaluation: controlled, ethically approved experiments with simulated opponents.
- Legal viability: study of integration within the institutional framework of persecution (AI Act Art. 50 exemption) with full DPIA.

11. Conclusions

This paper designed, implemented, and evaluated ScamAI, a comprehensive, two-stage training system that covers the entire anti-scam chain: from detection with interpretable Machine Learning to automated, *secure* response with Generative AI.

11.1. Answers to research questions

The paper answers the four questions posed in Section 1.4:

- (E1) Detection with a lightweight, interpretable model. Yes: even a set of 8 transparent attributes achieves useful separation capability (Random Forest, ROC-AUC 0.815, recall 0.65) on realistic, unbalanced data (7.7% fraud), detecting two-thirds of frauds. The complete failure of Naive Bayes confirms that managing the imbalance (via class_weight) is crucial.
- (Q2) Convincing but safe response. It is achieved by combining distinct personas by scam type, multi-turn coherence, and explicit exit security control that hides PII and blocks dangerous content before each exit.
- (E3) Security in a hostile environment. The main risk (prompt injection) is addressed with data/command separation, explicit non-execution instruction, and exit control as a last line of defense — a "deep defense" strategy.
- (E4) Legal and ethical framework. The analysis demonstrated that an actual deployment would only be defensible within an institutionalized framework for prosecuting crime, with a legal basis under GDPR, compliance with Article 50 of the AI Act, and respect for cybercriminal law.

11.2. Contribution

The contribution is primarily instructive: a complete, reproducible, and offline-possible example of the integration of Machine Learning, Generative AI, and Responsible AI principles (output controls, data/command separation, human-in-the-loop), accompanied by a thorough legal and ethical analysis that rarely accompanies technical tasks of this type.

11.3. Limitations

The findings should be interpreted under the constraints of the work: the shallow feature set (no TF-IDF/embeddings) puts an upper limit on performance; fraud-type detection is keyword-based and vulnerable to adversarial wording; security checks are rules (regex) and do not cover all possible leaks; And the responder's assessment is necessarily qualitative, as quantitative would require interaction with real cheaters – something the job deliberately avoids for legal and ethical reasons.

11.4. Concluding remarks

The overall conclusion is unified: the active defence against fraud is *Technical* feasible, but *indissoluble* linked to GDPR, the Artificial Intelligence Act, cybercriminal law, and fundamental ethical commitments. A sustainable implementation, therefore, is not just a matter of engineering; it assumes an institutional framework, a legal basis, and ongoing human oversight - principles that ScamAI adopts, in simulated form, already from its design. As fraudsters increasingly leverage Generative AI, the need for equally intelligent, yet responsible, defensive approaches will only become more imperative.

12. Bibliography

12.1. Scientific literature

[1] M. Sahami, S. Dumais, D. Heckerman, E. Horvitz, «A Bayesian Approach to Filtering Junk E-Mail», *AAAI Workshop on Learning for Text Categorization*, 1998.

[2] I. Androutsopoulos, J. Koutsias, K. V. Chandrinos, G. Paliouras, C. D. Spyropoulos, «An Evaluation of Naive Bayesian Anti-Spam Filtering», *ECML Workshop on Machine Learning in the New Information Age*, 2000.

[3] E. G. Dada, J. S. Bassi, H. Chiroma, S. M. Abdulhamid, A. O. Adetunmbi, O. E. Ajibuwa, «Machine learning for email spam filtering: review, approaches and open research problems», *Heliyon*, 5(6), e01802, 2019.

[4] A. Karim, S. Azam, B. Shanmugam, K. Kannoorpatti, M. Alazab, «A Comprehensive Survey for Intelligent Spam Email Detection», *IEEE Access*, 7, 168261–168295, 2019.

[5] L. Breiman, «Random Forests», *Machine Learning*, 45(1), 5–32, 2001.

[6] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, «SMOTE: Synthetic Minority Over-sampling Technique», *Journal of Artificial Intelligence Research*, 16, 321–357, 2002.

[7] M. A. Uddin, M. Mahiuddin, I. H. Sarker, «An Explainable Transformer-based Model for Phishing Email Detection: A Large Language Model Approach», arXiv:2402.13871, 2024.

[8] S. Jamal, H. Wimmer, I. H. Sarker, «An Improved Transformer-based Model for Detecting Phishing, Spam, and Ham: A Large Language Model Approach», arXiv:2311.04913, 2023.

[9] R. B. Cialdini, *Influence: The Psychology of Persuasion*, Revised Edition, Harper Business, 2007.

[10] D. Gragg, «A Multi-Level Defense Against Social Engineering», *SANS Institute Reading Room*, 2003.

[11] F. Stajano, P. Wilson, «Understanding scam victims: Seven principles for systems security», *Communications of the ACM*, 54(3), 70–75, 2011.

[12] A. Ferreira, L. Coventry, G. Lenzini, «Principles of Persuasion in Social Engineering and Their Use in Phishing», *Human Aspects of Information Security, Privacy and Trust (HCII)*, LNCS 9190, 36–47, 2015.

[13] C. Herley, «Why do Nigerian Scammers Say They are from Nigeria?», *Workshop on the Economics of Information Security (WEIS)*, 2012.

[14] M. Edwards, C. Peersman, A. Rashid, «Scamming the scammers: towards automatic detection of persuasion in advance fee frauds», *Proc. 26th International Conference on World Wide Web Companion (WWW '17 Companion)*, 1291–1299, 2017.

[15] A. Zingerle, L. Kronman, «Humiliating Entertainment or Social Activism? Analyzing Scambaiting Strategies Against Online Advance Fee Fraud», *International Conference on Cyberworlds (IEEE)*, 352–355, 2013.

[16] L. Tuovinen, J. Röning, «Baits and Beatings: Vigilante Justice in Virtual Communities», *Proc. Conference on Computer Ethics: Philosophical Enquiry (CEPE)*, 397–405, 2007.

[17] T. Sorell, «Scambaiting on the Spectrum of Digilantism», *Criminal Justice Ethics*, 38(3), 153–175, 2019.

[18] A. S. Ross, L. Logi, «"Hello, this is Martha": Interaction dynamics of live scambaiting on Twitch», *Convergence*, 27(6), 1789–1810, 2021.

[19] P. Bajaj, M. Edwards, «Automatic Scam-Baiting Using ChatGPT», arXiv:2309.01586, 2023.

[20] Netsafe, «Re:Scam» — automated email scambaiting bot, 2017.

[21] B. Acharya, D. Sautter, M. Saad, T. Holz, «ScamChatBot: An End-to-End Analysis of Fake Account Recovery on Social Media via Chatbots», arXiv:2412.15072, 2024.

[22] «The Imitation Game: Using Large Language Models as Chatbots to Combat Chat-Based Cybercrimes (LURE)», arXiv:2512.21371, 2025.

[23] N. Vogler et al., «Victim as a Service: Designing a System for Engaging with Interactive Scammers (CHATTERBOX)», arXiv:2510.23927, 2025.

[24] H. Siadati, H. Jafarian, S. Jafarikhah, «"Send to which account?" Evaluation of an LLM-based Scambaiting System», arXiv:2509.08493, 2025.

[25] I. Hossain, S. Puppala, M. J. Alam, S. Talukder, «AI-in-the-Loop: Privacy Preserving Real-Time Scam Detection and Conversational Scambaiting by Leveraging LLMs and Federated Learning», arXiv:2509.05362, 2025.

[26] A. Askell, Y. Bai, A. Chen, et al., «A General Language Assistant as a Laboratory for Alignment», arXiv:2112.00861, 2021.

[27] F. Perez, I. Ribeiro, «Ignore Previous Prompt: Attack Techniques For Language Models», *NeurIPS ML Safety Workshop*, 2022.

[28] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, «Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection», *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 79–90, 2023.

[29] "Prompt Injection Attacks on Large Language Models: A Survey of Attack Methods, Root Causes, and Defense Strategies", *Computers, Materials & Continua*, 2026.

[30] F. W. Liu, C. Hu, «Exploring Vulnerabilities and Protections in Large Language Models: A Survey», arXiv:2406.00240, 2024.

[31] OWASP Foundation, «OWASP Top 10 for Large Language Model Applications», 2025.

12.2. Legal instruments

[32] Regulation (EU) 2016/679 (General Data Protection Regulation — GDPR).

[33] Regulation (EU) 2024/1689 (AI Act), in particular Article 50.

[34] Directive (EU) 2022/2555 (NIS2).

[35] Directive 2013/40/EU on attacks against information systems (OJ L 218, 14.8.2013).

[36] Directive (EU) 2016/680 (Law Enforcement Directive — LED).

[37] Council of Europe Convention on Cybercrime (Budapest Convention), 2001.

[38] Law 4624/2019 "Personal Data Protection Authority..." (Government Gazette A΄137/29.08.2019).

[39] Law 3471/2006 on data protection in electronic communications (ePrivacy).

[40] Article 9A of the Constitution of Greece (revised 2001).

12.3. Additional reports (active defense & fraud)

[41] W. Chen, F. Wang, M. Edwards, "Active countermeasures for email fraud", *Proc. 8th IEEE European Symposium on Security and Privacy (EuroS&P),* 2023.

[42] M. Canham, J. Tuthill, «Planting a poison SEAD: Using social engineering active defense (SEAD) to counter cybercriminals», *Proc. International Conference on Human-Computer Interaction (HCII)*, Springer, 48–57, 2022.

[43] R. Zbinden, O. Beaudet-Labrecque, F. Grandjean, C. Gobeil, L. Brunoni, D. Décary-Hétu, C. Cretu-Adatte, "Scambaiting as a preventive tool in the fight against cyberfrauds: the case of romance scams", *The Journal on Cybercrime & Digital Investigations*, 2023.

[44] I. Chadalavada, T. Huang, J. Staddon, «Distinguishing Scams and Fraud with Ensemble Learning», arXiv:2412.08680, 2024.

[45] X. W. Tan, K. See, S. Kok, «ScamGPT-J: Inside the Scammer's Mind, A Generative AI-Based Approach Toward Combating Messaging Scams», arXiv:2412.13528, 2024.

[46] B. Acharya, T. Holz, «An explorative study of pig butchering scams», arXiv:2412.15423, 2024.

[47] Federal Bureau of Investigation (FBI), Internet Crime Complaint Center (IC3), «2024 Internet Crime Report», 2025.

13. Annexes

13.1. Annex A - Full transcript multi-turn (mock provider)

Type of scam: nigerian_prince· 3 rounds; all with a clean security check. Full file: transcript_nigerian.md.

13.2. Annex B - Definitions of characteristics

See. §4.4 for the complete table of the 8 characteristics and their logic.

13.3. Annex C - Glossary

- **Scambaiting:** the deliberate engagement with a scammer to waste his time/resources.
- **Advance-fee fraud ('419'):** advance fee fraud.
- **Prompt injection:** attack where malicious commands are embedded in LLM input data.
- **PII:** Personally Identifiable Information.
- **DPIA:** Data Protection Impact Assessment.
- **ROC-AUC:** area below the ROC curve; measure of separation capacity.

13.4. Annex D - Distribution of team roles

Role	Competence

Person 1	Dataset & Preprocessing
Person 2	ML Classifier
Person 3	Generative AI Responder
Person 4	Pipeline & Demo UI
Person 5	Literature Review & Ethics
Person 6	Report & Coordination

13.5. Annex E - Selected Code Snippets

Three representative excerpts of the actual code are listed, so that the report also documents the implementation.

13.6. E.1 Attribute extraction (preprocessing/preprocess.py)

```
def extract_features(text: str) -> dict:
    cleaned = clean_text(text)
    tokens = word_tokenize(cleaned)
    tokens_no_stop = [t for t in tokens if t not in STOP_WORDS]

    word_count = len(tokens)
    unique_words = len(set(tokens_no_stop))
    avg_word_length = (
        sum(len(w) for w in tokens_no_stop) / len(tokens_no_stop)
        if tokens_no_stop else 0
    )

    scam_keyword_count = sum(1 for kw in SCAM_KEYWORDS if kw in cleaned)

    # Capitalization signals (scammers SHOUTING)
    original_words = text.split()
    caps_ratio = (
```

```
        sum(1 for w in original_words if w.isupper() and len(w) > 2)
        / len(original_words) if original_words else 0
    )

    exclamation_count = text.count("!")
    question_count = text.count("?")

    # Money mentions — in the ORIGINAL text (before deducting $ and digits)
    money_mentions = len(re.findall(
        r"\$[\d,]+|\d+\s*(?:million|billion|usd|dollars|gbp)",
        text, flags=re. IGNORECASE))

    return {
        "word_count": word_count, "unique_words": unique_words,
        "avg_word_length": avg_word_length,
        "scam_keyword_count": scam_keyword_count, "caps_ratio": caps_ratio,
        "exclamation_count": exclamation_count,
        "question_count": question_count, "money_mentions": money_mentions,
    }
```

The comment in the line of money_mentions Documents the bug fix described in §4.4: the calculation is done in the original text, before the cleanup removes symbols and digits.

13.6.1. E.2 PII Detection Patterns (responder/responder.py)

```
_PII_PATTERNS = {
    "iban":        re.compile(r"\b[A-Z]{2}\d{2}[A-Z0-9]{10,30}\b"),
    "credit_card": re.compile(r"\b(?:\d[ -]?){13,16}\b"),
    "ssn":         re.compile(r"\b\d{3}-\d{2}-\d{4}\b"),
    "btc_wallet":  re.compile(r"\b(?:bc1|[ 13])[a-km-zA-HJ-NP-Z1-9]{25,39}\b"),
```

```
    "eth_wallet":  re.compile(r"\b0x[a-fA-F0-9]{40}\b"),
    "email":       re.compile(r"\b[\w.+-]+@[\w-]+\.[ \w.-]+\b"),
    "phone":       re.compile(r"(?<!\w)(?:\+?\d[\d\s().-]{8,}\d)(?! \w)"),
}
```

The patterns cover the main categories of real information that should never be leaked in a response: international bank numbers (IBANs), cards, social security numbers, cryptocurrency wallets, emails, and phones.

13.6.2. E.3 Exit safety check (responder/responder.py)

```
def safety_check(reply: str) -> dict:
 issues = []
 sanitized = reply or ""#

    1) Threats/abuse → HARD FAIL: all response is rejected.
    if _ABUSE_PATTERNS.search(sanitized):
        return {"safe": False, "issues": ["abusive_content"],
                "sanitized_reply": _SAFE_FALLBACK}

    # 2) Real appointment/address → escalation risk → redact + flag.
    if _MEETUP_PATTERNS.search(sanitized):
        issues.append("meetup_or_address")
        sanitized = _MEETUP_PATTERNS.sub("[REDACTED — no real meetings]", sanitized)

    # 3) PII / banking / wallets → redact + flag.
    for name, pat in _PII_PATTERNS.items():
        if pat.search(sanitized):
            issues.append(f"pii_{name}")
            sanitized = pat.sub(f"[REDACTED_{name.upper()}]", sanitized)
```

```
    # 4) Empty/degenerate response → safe fallback.
    if not sanitized.strip():
        issues.append("empty_reply")
        sanitized = _SAFE_FALLBACK

    return {"safe": len(issues) == 0, "issues": issues,
            "sanitized_reply": sanitized}
```

The structure captures the risk hierarchy: threats lead to *Complete rejection* (hard fail), while PII and appointments *concealed* (redaction) so that the conversation can continue safely. The return of a structured dict (safe/issues/sanitized_reply) allows the pipeline and UI to display the security status.